Laboratório Prolog · Aula 13

Gabriel Galhardone Felipe · RA 24021292-2 · Turma Esoft B
Gerado em 20/10/2026, 20:16:43 · tempo registrado no cronômetro: 30:05

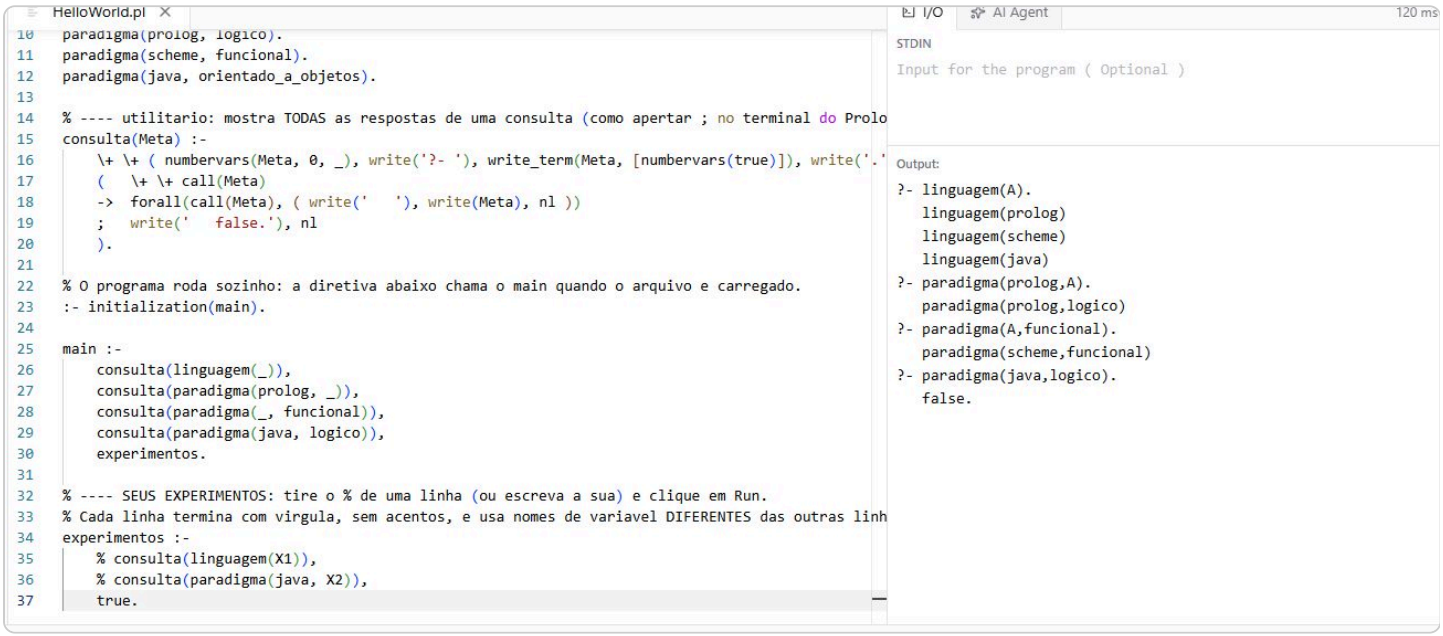
Preparação: Programar descrevendo, não instruindo

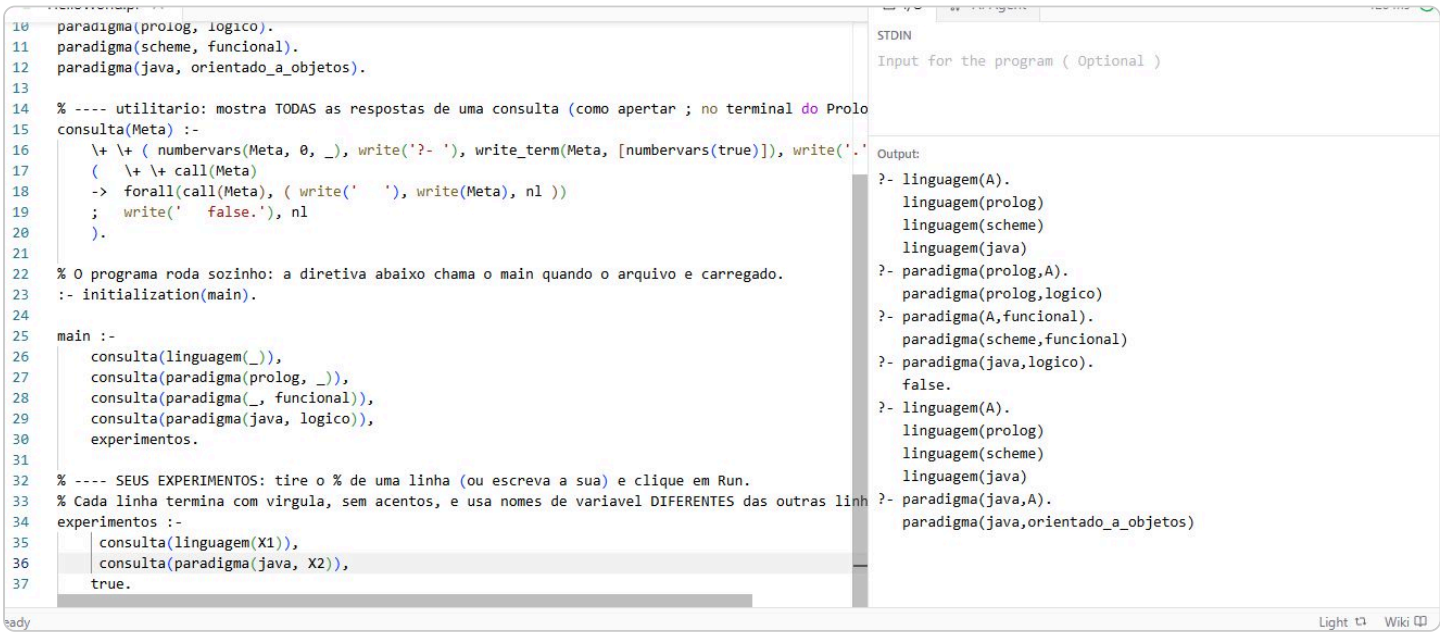
Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

A primeira consulta paradigma(L, funcional) vai mostrar todas as linguagens cadastradas que possuem o paradigma funcional, substituindo L por cada resposta encontrada.
A segunda consulta paradigma(java, logico) vai retornar true se Java estiver cadastrado como lógico; caso contrário, false.

Saída do ola.pl

```
saida normal e saida sem a porcentagem
```





0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) pergunta qual é o paradigma do Prolog, enquanto paradigma(prolog, logico) verifica se o paradigma do Prolog é lógico

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Significa que o programa não possui esse fato declarado, pois Java está associado ao paradigma orientado a objetos, e não ao lógico.

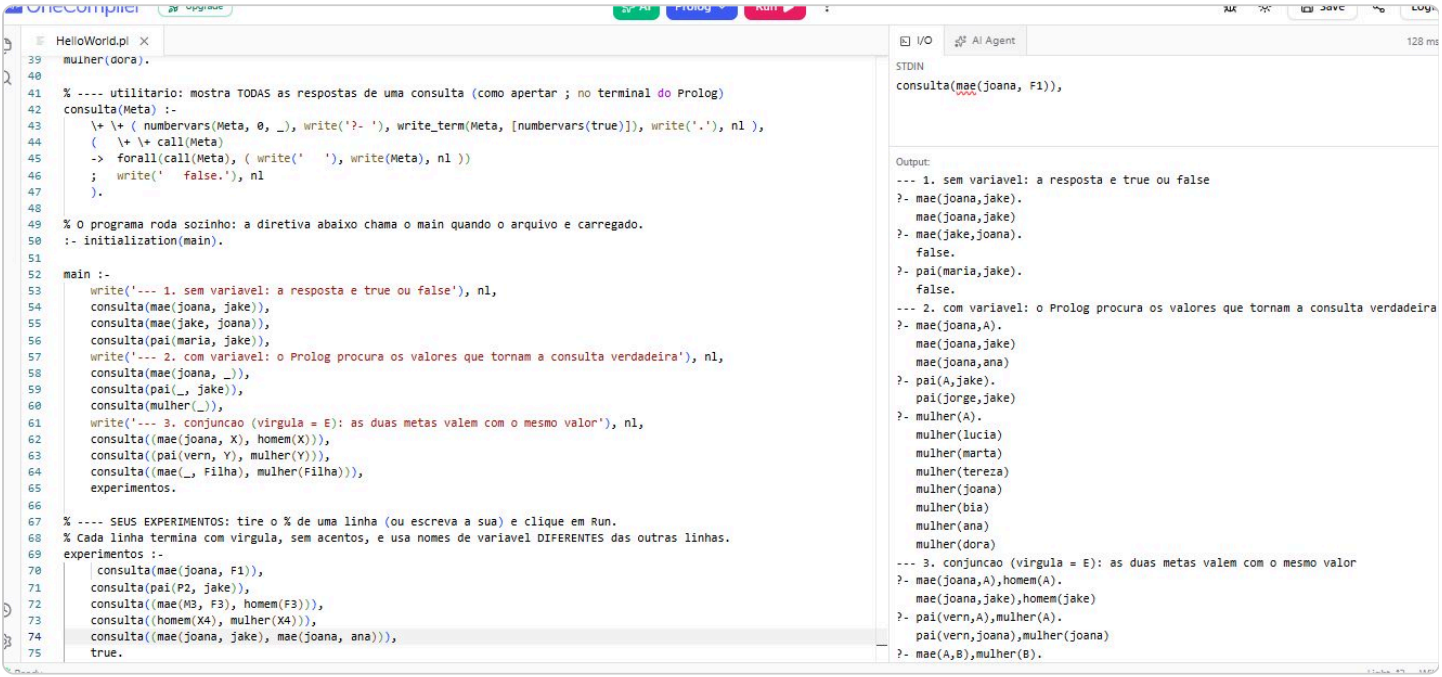
Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) terá uma resposta para cada fato mulher cadastrado na base. Já (mae(M, F), mulher(F)) retornará os pares em que M é mãe de F e F é mulher, considerando os fatos declarados no programa.

Saída do familia.pl

```
exercicio
```



1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

A consulta diz quais mães tem filhas mulheres e teve 3 respostas.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

O Prolog considera falso tudo que não está registrado na base. Isso pode ser um problema na vida real porque uma informação pode ser verdadeira, mas ainda não ter sido cadastrada no sistema.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

Apareceram na mesma ordem em que as mulheres foram colocadas no código, de cima para baixo.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake) retorna os tios de Jake, e primo(leo, X) retorna os primos de Leo. As respostas exatas dependem das regras e dos fatos definidos no código da atividade.

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Vão falhar f(X, X) = f(a, b), porque X não pode ser a e b ao mesmo tempo; [A|B] = [], porque uma lista vazia não tem primeiro elemento; e 2 + 3 = 5, porque o Prolog não calcula automaticamente expressões matemáticas na unificação.

Saídas de regras.pl e unificacao.pl

```
resp
```



2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Ela precisa de duas cláusulas para encontrar tanto os parentes diretos quanto os parentes mais distantes. O caso base verifica se alguém é progenitor direto, e o caso recursivo continua procurando os ancestrais dos progenitores até encontrar todos.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, o Prolog passa a considerar a própria pessoa como irmã dela mesma. Essa condição é necessária para garantir que X e Y sejam pessoas diferentes.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Na primeira, não unifica porque X teria que ser a e b ao mesmo tempo. Na segunda, unifica porque X fica com o valor a e Y também pode ser a, deixando os dois lados iguais.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

No Prolog, a mesma regra pode responder às duas perguntas porque ele procura os valores que tornam a consulta verdadeira. Já em Java ou Scheme, normalmente precisaríamos escrever funções ou passar parâmetros específicos para cada consulta.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

Vai responder com 4 possibilidades de divisão da lista: X = [] e Y = [1,2,3]; X = [1] e Y = [2,3]; X = [1,2] e Y = [3]; X = [1,2,3] e Y = [].

Saída do listas.pl

```
--- membro: serve para testar E para gerar
?- membro(b, [a, b, c]).
   membro(b, [a, b, c])
?- membro(z, [a, b, c]).
   false.
?- membro(_, [a, b, c]).
   membro(a, [a, b, c])
   membro(b, [a, b, c])
   membro(c, [a, b, c])
--- tamanho
?- tamanho([a, b, c], _).
   tamanho([a, b, c], 3)
?- tamanho([], _).
   tamanho([], 0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1, 2], [3, 4], _).
   concatena([1, 2], [3, 4], [1, 2, 3, 4])
?- concatena(_, [3], [1, 2, 3]).
   concatena([1, 2], [3], [1, 2, 3])
?- concatena([1], _, [1, 2, 3]).
   concatena([1], [2, 3], [1, 2, 3])
?- concatena(_, _, [1, 2, 3]).
   concatena([], [1, 2, 3], [1, 2, 3])
   concatena([1], [2, 3], [1, 2, 3])
   concatena([1, 2], [3], [1, 2, 3])
   concatena([1, 2, 3], [], [1, 2, 3])
--- inverte
?- inverte([1, 2, 3], _).
   inverte([1, 2, 3], [3, 2, 1])
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

O caso base é tamanho([], 0), quando a lista está vazia. O passo recursivo tira um elemento e soma 1 ao tamanho da cauda. Para [a, b, c], fica pendente somar 1 a cada chamada até chegar à lista vazia.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Tem 4 respostas porque a lista pode ser dividida em duas partes de 4 formas diferentes. Já no Scheme, o anexar apenas junta duas listas, enquanto no Prolog a mesma relação também consegue descobrir quais são as partes.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

Isso acontece por causa da recursão e do backtracking, que fazem o Prolog percorrer a lista e encontrar cada elemento sem precisar de um laço.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3 deixa X como a expressão 2+3, sem calcular. Já X is 2 + 3 calcula a expressão e retorna X = 5. A consulta fatorial(N, 120) não funciona para descobrir N, porque a regra usa aritmética que precisa de valores já conhecidos.

Saída do aritmetica.pl

```
--- = unifica, is calcula
?- _ = 2+3.
   _ = 2+3
?- _ is 2+3.
   5 is 2+3
?- 5 =:= 2+3.
   true
?- 2+3 = 5.
   false.
?- _ is 7 mod 3.
   1 is 7 mod 3
?- _ is 7//2.
   3 is 7//2
?- _ is 2^10.
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5, _).
   fatorial(5, 120)
?- fatorial(0, _).
   fatorial(0, 1)
?- fib(10, _).
   fib(10, 55)
?- soma([1, 2, 3, 4], _).
   soma([1, 2, 3, 4], 10)
--- aritmetica nao e relacional
   ERRO: instantiation_error
   ERRO: instantiation_error
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

O = serve para comparar ou unir estruturas, sem fazer cálculos, como X = 2 + 3. O is calcula o resultado, por exemplo, X is 2 + 3, que dá 5. Já o =:= compara os valores numéricos, como 5 =:= 2 + 3, que retorna verdadeiro.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

fatorial(N, 120) dá erro porque a aritmética em Prolog precisa que as variáveis usadas nos cálculos estejam definidas. Como N está livre, a operação N > 0 gera erro. Já concatena(X, [3], [1, 2, 3]) funciona porque a unificação permite descobrir os valores das variáveis comparando as listas, sem precisar realizar cálculos aritméticos.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Os dois usam recursão e um caso base para calcular o fatorial. A diferença é que, em Haskell, a função retorna o resultado diretamente, enquanto, em Prolog, o resultado é obtido por meio de relações e unificação. Além disso, o Prolog depende da ordem das operações aritméticas, usando is para calcular o resultado.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro1(X) ou solteiro2(X) dará false dependendo da ordem das metas e do uso da negação no código. Para confirmar qual delas, preciso ver as regras completas.

maximo2(5, 3, M) dá duas respostas possíveis: M = 5 e M = 3, porque a segunda cláusula não tem teste.

maximo3(5, 3, 3) dá false, porque a primeira cláusula exige M = 5 quando X >= Y, e o corte ! impede tentar outras cláusulas.

Saída de controle.pl e mensagem de ancestral_ruim

```
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
   resposta: mesmo_gosto(ana,caio)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(caio,ana)
   tenta caio e bia
   tenta caio e caio
--- B) negacao: a ordem das metas muda a resposta
?- solteiro1(_).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- solteiro2(_).
   false.
--- C) corte
?- maximo1(5,3,_).
   maximo1(5,3,5)
?- maximo2(5,3,_).
   maximo2(5,3,5)
   maximo2(5,3,3)
?- maximo3(5,3,_).
   maximo3(5,3,5)
?- maximo1(5,3,3).
   false.
?- maximo3(5,3,3).
   false.


ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)
Fatal Error: local stack overflow
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores, existem colorações válidas; com 2 cores, não existe nenhuma (\(0\)), porque PR, SP e MS fazem fronteira entre si e formam um triângulo, exigindo três cores diferentes.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],_).
```

```
   false.
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

Na tentativa ana e ana, a meta X \== Y falhou, porque X e Y eram iguais. Então, o Prolog voltou ao último ponto de escolha e tentou outro valor para Y. Continuou testando as combinações até encontrar as duas respostas válidas: ana e caio e caio e ana, pois ambos gostam de café.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

solteiro2(X) dá false porque a negação \+ casado(X) é executada antes de X receber um valor, então falha. Já solteiro1(X) primeiro busca os homens e depois verifica se não há fato dizendo que são casados. Ele afirma que Osvaldo é solteiro porque o programa não tem nenhum fato casado(osvaldo). Ser pai de alguém não significa ser casado.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O corte não protegeu porque a primeira cláusula unifica o terceiro argumento com X antes de executar o corte. Para corrigir, faça o teste e o corte antes da unificação do resultado:
maximo3(X, Y, M) :- X >= Y, !, M = X.
maximo3(_, Y, Y).

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim estoura a pilha porque chama a si mesma antes de verificar se existe um progenitor, entrando em recursão infinita. Já ancestral_bom verifica primeiro a relação progenitor e só depois faz a chamada recursiva, avançando em direção ao caso base. Assim, a ordem das metas permite que ancestral_bom termine corretamente.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

O programa encontrou 24 colorações com 3 cores, 0 com 2 cores e 1.728 com 4 cores. Ele é declarativo porque descreve as condições para uma coloração válida, sem definir passo a passo como encontrar as soluções. O Prolog usa o backtracking para realizar a busca automaticamente.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No paradigma funcional, como em Scheme com anexar, a função recebe listas e devolve uma nova lista como resultado. No paradigma lógico, como em Prolog com concatena, descrevemos uma relação entre listas, permitindo descobrir diferentes argumentos conforme a consulta.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

Uma deficiência é a dependência da ordem das metas, observada em ancestral_ruim, que entrou em recursão infinita e estourou a pilha. Outra é a aritmética não relacional, observada em fatorial(N, 120), que gerou erro porque N não estava definido.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas de inteligência artificial, sistemas especialistas e busca de soluções com regras e restrições, pois ele permite descrever fatos e relações. Não escolheria para sistemas comerciais grandes, como aplicativos bancários, pois Java oferece mais controle sobre o fluxo do programa e é mais adequado para desenvolver aplicações complexas.

Exercícios opcionais

Opcionais: resultado dos testes

```
Exercicio 1
   ok      irma(ana, jake)
   ok      irma(tereza, vern)
   ok      NAO irma(jake, ana)
   ok      tia(tereza, joana)
   ok      tia(joana, leo)
   ok      NAO tia(joana, jake)
Exercicio 2
   ok      ultimo([a,b,c], c)
   ok      ultimo([7], 7)
   ok      NAO ultimo([a,b,c], b)
Exercicio 3
   ok      soma_quadrados([1,2,3], 14)
   ok      soma_quadrados([], 0)
   ok      pares([1,2,3,4,5,6], [2,4,6])
   ok      pares([1,3], [])
Exercicio 4
   ok      mdc(12, 18, 6)
   ok      mdc(17, 5, 1)
   ok      mdc(10, 0, 10)
   ok      potencia(2, 10, 1024)
   ok      potencia(5, 0, 1)
Exercicio 5
   ok      caminho(curitiba, foz)
   ok      caminho(joinville, cascavel)
   ok      NAO caminho(foz, curitiba)
   ok      NAO caminho(curitiba, paris)
Exercicio 6
   ok      3 cores: 12 coloracoes
   ok      2 cores: 2 coloracoes
   ok      1 cor: nenhuma
Exercicio 7
   ok      colega(ana, bruno)
   ok      NAO colega(ana, carla)
   ok      NAO colega(ana, ana)
   ok      ganha_mais(ana, bruno)
   ok      NAO ganha_mais(elisa, ana)
   ok      total_depto(ti, 12500)
   ok      total_depto(rh, 9000)
```

E1 Regras de família: código escrito

```
irma(X, Y) :-
    irmao(X, Y),
    mulher(X).

tia(X, Y) :-
    irma(X, P),
    progenitor(P, Y).
```

E2 Listas: código escrito

```
ultimo([X], X).
ultimo([_|T], X) :-
    ultimo(T, X).
```